



dials (version 1.2.0)

grid_regular: Create grids of tuning parameters

Description

Random and regular grids can be created for any number of parameter objects.

Usage

```
grid_regular(x, ..., levels = 3, original = TRUE, filter = NULL)

# S3 method for parameters
grid_regular(x, ..., levels = 3, original = TRUE, filter = NULL)

# S3 method for list
grid_regular(x, ..., levels = 3, original = TRUE, filter = NULL)

# S3 method for param
grid_regular(x, ..., levels = 3, original = TRUE, filter = NULL)

# S3 method for workflow
grid_regular(x, ..., levels = 3, original = TRUE, filter = NULL)

grid_random(x, ..., size = 5, original = TRUE, filter = NULL)

# S3 method for parameters
grid_random(x, ..., size = 5, original = TRUE, filter = NULL)

# S3 method for list
grid_random(x, ..., size = 5, original = TRUE, filter = NULL)

# S3 method for param
grid_random(x, ..., size = 5, original = TRUE, filter = NULL)

# S3 method for workflow
grid_random(x, ..., size = 5, original = TRUE, filter = NULL)
```

Value

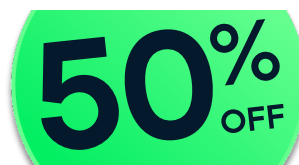
A tibble. There are columns for each parameter and a row for every parameter combination.

Arguments

x	A <code>`param`</code> object, list, or <code>`parameters`</code> .
...	One or more <code>`param`</code> objects (such as <code>`mtry()`</code> or <code>`penalty()`</code>). None of the objects can have <code>`unknown()`</code> values in the parameter ranges or values.
levels	An integer for the number of values of each parameter to use to make the regular grid. <code>`levels`</code> can be a single integer or a vector of integers that is the same length as the number of parameters in <code>`...`</code> . <code>`levels`</code> can be a named integer vector, with names that match the id values of parameters.
original	A logical: should the parameters be in the original units or in the transformed space (if any)?
filter	A logical: should the parameters be filtered prior to generating the grid. Must be a single expression referencing parameter names that evaluates to a logical vector.
size	A single integer for the total number of parameter value combinations returned for the random grid. If duplicate combinations are generated from this size, the smaller, unique set is returned.

Details

NEW YEAR
NEW SKILLS



Buy Now > X

mixture()

```
## Proportion of Lasso Penalty (quantitative)
## Range: [0, 1]

set.seed(283)
mix_grid_1 <- grid_random(mixture(), size = 1000)
range(mix_grid_1$mixture)

## [1] 0.001490161 0.999741096
```

However, in some cases, the ``parsnip`` and ``recipe`` packages overrides the default ranges for specific models and preprocessing steps. If the grid function uses a ``parameters`` object created from a model or recipe, the ranges may have different defaults (specific to those models). Using the example above, the ``mixture`` argument above is different for ``glmnet`` models:

```
library(parsnip)
library(tune)

# When used with glmnet, the range is [0.05, 1.00]
glmn_mod <-
  linear_reg(mixture = tune()) %>%
  set_engine("glmnet")

set.seed(283)
mix_grid_2 <- grid_random(extract_parameter_set_dials(glmn_mod), size = 1000)
range(mix_grid_2$mixture)

## [1] 0.05141565 0.99975404
```

Examples

Run this code

```
# filter arg will allow you to filter subsequent grid data frame based on some condition.
p <- parameters(penalty(), mixture())
grid_regular(p)
grid_regular(p, filter = penalty <= .01)

# Will fail due to unknowns:
# grid_regular(mtry(), min_n())

grid_regular(penalty(), mixture())
grid_regular(penalty(), mixture(), levels = 3:4)
grid_regular(penalty(), mixture(), levels = c(mixture = 4, penalty = 3))
grid_random(penalty(), mixture())
```

Run the code above in your browser using [DataCamp Workspace](#)

Powered by [DataCamp](#)